Assignment 8: Basic SQL Queries and Aggregate Functions

Hospital Management System - WHERE, ORDER BY, GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX

1. Objective

This assignment demonstrates basic SQL retrieval and manipulation on the Hospital Management System database loaded in Assignment 7: filtering with WHERE, sorting with ORDER BY, grouping with GROUP BY, filtering groups with HAVING, and the aggregate functions COUNT, SUM, AVG, MIN and MAX. Every query was executed on the database and the actual output is shown beneath it.

2. Filtering with WHERE

Query 1: Female patients born after 1985

```
SELECT patient_id, name, DOB, Gender
FROM PATIENTS
WHERE Gender = 'Female' AND DOB > '1985-01-01';
```

Output

```
+------------+-----------------+------------+--------+
| patient_id | name            | DOB        | Gender |
+------------+-----------------+------------+--------+
|       1002 | Anitha Rao      | 1992-07-25 | Female |
|       1004 | Sneha Kulkarni  | 2001-01-18 | Female |
|       1006 | Divya Hegde     | 1995-05-14 | Female |
|       1010 | Pooja Sharma    | 1988-02-27 | Female |
+------------+-----------------+------------+--------+
```

4 row(s) returned

Explanation: WHERE keeps only rows that satisfy both conditions joined by AND; here female patients born after 1 January 1985.

Query 2: Upcoming scheduled appointments in a date range

```
SELECT appointment_id, appointment_date, appointment_status, patient_id, doctor_id
FROM APPOINTMENT
WHERE appointment_status = 'Scheduled'
  AND appointment_date BETWEEN '2026-10-24' AND '2026-11-05';
```

Output

```
+---------------+-----------------+--------------------+------------+-----------+
| appointment_id | appointment_date | appointment_status | patient_id | doctor_id |
+---------------+-----------------+--------------------+------------+-----------+
|          5017 | 2026-10-26      | Scheduled          |       1003 |       102 |
|          5018 | 2026-10-28      | Scheduled          |       1007 |       106 |
|          5019 | 2026-10-30      | Scheduled          |       1009 |       101 |
|          5020 | 2026-11-02      | Scheduled          |       1004 |       108 |
+---------------+-----------------+--------------------+------------+-----------+
```

4 row(s) returned

Explanation: BETWEEN selects an inclusive range. All four future appointments fall in the window.

Query 3: Medicines matching a name pattern or a price condition

```
SELECT medicine_id, medicine_name, price, stock_quantity
FROM MEDICINES
WHERE medicine_name LIKE 'A%' OR price > 40;
```

Output

```
+-------------+--------------------+--------+----------------+
| medicine_id | medicine_name      | price  | stock_quantity |
+-------------+--------------------+--------+----------------+
|         202 | Amoxicillin 500mg  |   8.00 |           2500 |
|         203 | Atorvastatin 20mg  |  12.00 |           1800 |
|         204 | Amlodipine 5mg     |   4.50 |           3000 |
|         206 | Aspirin 75mg       |   1.80 |           6000 |
|         208 | Ceftriaxone 1g Inj |  45.00 |            400 |
|         209 | Insulin Glargine   | 650.00 |             80 |
+-------------+--------------------+--------+----------------+
```

6 row(s) returned

Explanation: LIKE 'A%' matches names starting with A; OR adds the expensive medicines (price above 40).

Query 4: Admissions with no discharge date (patients currently admitted)

```
SELECT admission_id, patient_id, room_number, admission_date
FROM ADMISSIONS
WHERE discharge_date IS NULL;
```

Output

```
+--------------+------------+-------------+----------------+
| admission_id | patient_id | room_number | admission_date |
+--------------+------------+-------------+----------------+
|         7008 |       1012 | B-315       | 2026-10-20     |
+--------------+------------+-------------+----------------+
```

1 row(s) returned

Explanation: NULL cannot be compared with =; IS NULL finds rows where the discharge date is missing, i.e. the patient is still in hospital.

3. Sorting with ORDER BY

Query 5: Doctors sorted by specialization, then name

```
SELECT doctor_id, doctor_name, specialization
FROM DOCTOR
ORDER BY specialization ASC, doctor_name ASC;
```

Output

```
+-----------+------------------+--------------------+
| doctor_id | doctor_name      | specialization     |
+-----------+------------------+--------------------+
|       101 | Dr. Anil Kumar   | Cardiologist       |
|       102 | Dr. Meera Nair   | Cardiologist       |
|       108 | Dr. Lakshmi Iyer | General Physician  |
|       109 | Dr. Mohammed Faiz | General Physician |
|       110 | Dr. Sunita Patil | Gynecologist       |
|       104 | Dr. Kavita Shetty | Neurologist       |
|       103 | Dr. Suresh Rao   | Neurologist        |
|       105 | Dr. Rajesh Gowda | Orthopedic Surgeon |
|       107 | Dr. Arjun Reddy  | Pediatrician       |
|       106 | Dr. Priya Menon  | Pediatrician       |
+-----------+------------------+--------------------+
```

10 row(s) returned

Explanation: Rows are sorted by the first column; ties are broken by the second.

Query 6: Five most expensive medicines

```
SELECT medicine_name, manufacturer, price

FROM MEDICINES

ORDER BY price DESC

LIMIT 5;
```

Output

```
+--------------------+--------------+--------+
| medicine_name      | manufacturer | price  |
+--------------------+--------------+--------+
| Insulin Glargine   | Biocon       | 650.00 |
| Ceftriaxone 1g Inj | Alkem        |  45.00 |
| Atorvastatin 20mg  | Dr. Reddys   |  12.00 |
| Clopidogrel 75mg   | Torrent      |   9.50 |
| Amoxicillin 500mg  | Sun Pharma   |   8.00 |
+--------------------+--------------+--------+
```

5 row(s) returned

Explanation: DESC sorts from highest to lowest; LIMIT 5 keeps the top five rows.

4. Aggregate Functions

Query 7: COUNT: rows and non-NULL values

```
SELECT COUNT(*) AS total_admissions, COUNT(discharge_date) AS discharged_admissions
FROM ADMISSIONS;
```

Output

```
+------------------+-----------------------+
| total_admissions | discharged_admissions |
+------------------+-----------------------+
|                8 |                     7 |
+------------------+-----------------------+
```

1 row(s) returned

Explanation: COUNT(*) counts all rows, while COUNT(column) ignores NULLs, so the difference (1) is the patient not yet discharged.

Query 8: SUM, AVG, MIN and MAX of bill amounts

```
SELECT COUNT(*) AS bills_count,
       SUM(total_amount) AS total_billed,
       AVG(total_amount) AS avg_bill,
       MIN(total_amount) AS lowest_bill,
       MAX(total_amount) AS highest_bill
FROM BILL;
```

Output

```
+-------------+--------------+--------------+-------------+--------------+
| bills_count | total_billed | avg_bill     | lowest_bill | highest_bill |
+-------------+--------------+--------------+-------------+--------------+
|           9 |    406750.50 | 45194.500000 |      800.00 |    120000.00 |
+-------------+--------------+--------------+-------------+--------------+
```

1 row(s) returned

Explanation: One summary row for the whole BILL table: number of bills, their total, average, smallest and largest amount.

Query 9: Aggregates on medicine stock and price

```
SELECT SUM(stock_quantity) AS total_stock, AVG(price) AS avg_price,
       MIN(price) AS lowest_price, MAX(price) AS highest_price
FROM MEDICINES;
```

Output

```
+-------------+-----------+--------------+---------------+
| total_stock | avg_price | lowest_price | highest_price |
+-------------+-----------+--------------+---------------+
|       28430 | 67.681818 |         1.80 |        650.00 |
+-------------+-----------+--------------+---------------+
```

1 row(s) returned

Explanation: Shows total units in stock and the price spread of the medicine catalogue.

5. Grouping with GROUP BY

Query 10: Number of appointments per status

```
SELECT appointment_status, COUNT(*) AS appointments
FROM APPOINTMENT
GROUP BY appointment_status
ORDER BY appointments DESC;
```

Output

```
+--------------------+--------------+
| appointment_status | appointments |
+--------------------+--------------+
| Completed          |           14 |
| Scheduled          |            4 |
| Cancelled          |            2 |
+--------------------+--------------+
```

3 row(s) returned

Explanation: GROUP BY forms one group per status and COUNT(*) counts rows in each group.

Query 11: Billing summary by payment status

```
SELECT payment_status, COUNT(*) AS bills, SUM(total_amount) AS total_billed, AVG(total_amount) AS
avg_bill
FROM BILL
GROUP BY payment_status;
```

Output

payment_status	bills	total_billed	avg_bill
Paid	6	219850.50	36641.75
Partial	1	48000.00	48000.00
Pending	2	138900.00	69450.00

3 row(s) returned

Explanation: Combines grouping with SUM and AVG to show how much money is paid, partially paid or pending.

6. Filtering Groups with HAVING

Query 12: Doctors with at least 3 appointments

```
SELECT doctor_id, COUNT(*) AS appointments
FROM APPOINTMENT
GROUP BY doctor_id
HAVING COUNT(*) >= 3;
```

Output

```
+-----------+--------------+
| doctor_id | appointments |
+-----------+--------------+
|       101 |            4 |
|       108 |            4 |
+-----------+--------------+
```

2 row(s) returned

Explanation: HAVING filters *groups* after aggregation (WHERE cannot use COUNT). Only doctors 101 and 108 reach three appointments.

Query 13: Patients whose total billing exceeds 50,000

```
SELECT patient_id, COUNT(*) AS bills, SUM(total_amount) AS total_billed
FROM BILL
GROUP BY patient_id
HAVING SUM(total_amount) > 50000
ORDER BY total_billed DESC;
```

Output

```
+------------+-------+--------------+
| patient_id | bills | total_billed |
+------------+-------+--------------+
|       1005 |     1 |    120000.00 |
|       1003 |     1 |     85000.00 |
|       1009 |     1 |     62500.50 |
|       1010 |     1 |     54300.00 |
+------------+-------+--------------+
```

4 row(s) returned

Explanation: Groups bills by patient, then keeps only high-value patients.

Query 14: WHERE and HAVING together: completed appointments per doctor

```
SELECT doctor_id, COUNT(*) AS completed
FROM APPOINTMENT
WHERE appointment_status = 'Completed'
GROUP BY doctor_id
HAVING COUNT(*) >= 2;
```

Output

```
+-----------+-----------+
| doctor_id | completed |
+-----------+-----------+
|       101 |         3 |
|       108 |         2 |
|       110 |         2 |
+-----------+-----------+
```

3 row(s) returned

Explanation: WHERE first removes non-completed rows, GROUP BY groups what remains, HAVING keeps doctors with two or more completed visits.

7. Data Manipulation (INSERT, UPDATE, DELETE)

Manipulation statements are run inside a transaction and rolled back at the end, so the sample data stays unchanged for the later assignments.

Query 17: INSERT, UPDATE and DELETE with verification

```
INSERT INTO APPOINTMENT (appointment_id, appointment_date, patient_id, doctor_id)
VALUES (5021, '2026-11-10', 1013, 107);

UPDATE APPOINTMENT SET appointment_status = 'Completed' WHERE appointment_id = 5017;

UPDATE MEDICINES SET stock_quantity = stock_quantity - 30 WHERE medicine_id = 206;

DELETE FROM APPOINTMENT WHERE appointment_status = 'Cancelled';

SELECT appointment_status, COUNT(*) AS appointments FROM APPOINTMENT GROUP BY appointment_status;
```

```
+--------------------+--------------+
| appointment_status | appointments |
+--------------------+--------------+
| Completed          |           15 |
| Scheduled          |            4 |
+--------------------+--------------+
```

2 row(s) returned

```
SELECT medicine_id, medicine_name, stock_quantity FROM MEDICINES WHERE medicine_id = 206;
```

```
+-------------+---------------+----------------+
| medicine_id | medicine_name | stock_quantity |
+-------------+---------------+----------------+
|         206 | Aspirin 75mg  |           6000 |
+-------------+---------------+----------------+
```

1 row(s) returned

Explanation: a new appointment was inserted (status takes the DEFAULT 'Scheduled'), appointment 5017 was marked Completed, 30 units of Aspirin were deducted from stock, and the two cancelled appointments were deleted. The grouped counts confirm the changes. ROLLBACK then restored the original data.